RELATÓRIO - IA para Desenvolvimento + Revisão de Estruturas de Dados I

Curso: Engenharia de Software

Disciplina: Algoritimos avançados

Equipe: Cauê Fernandes, Luiz Fernando, Walter Theodoro

1. Two Sum

Link: https://leetcode.com/problems/two-sum/

Estrutura principal utilizada: Array + HashMap

Ideia da solução

Percorrer o array uma vez armazenando valores já vistos em um HashMap para encontrar complemento.

Justificativa da estrutura

Array permite acesso sequencial eficiente. HashMap permite busca O(1).

Código

```
def two_sum(nums, target):
    mapa = {}
    for i, num in enumerate(nums):
        complemento = target - num
        if complemento in mapa:
            return [mapa[complemento], i]
        mapa[num] = i
```

Complexidade

Tempo: O(n)
Memória: O(n)

Testes adicionais (2)

```
print(two_sum([3,3],6))
print(two_sum([2,7,11,15],9))
```

2. Best Time to Buy and Sell Stock

Link: https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

Estrutura principal utilizada: Array

Ideia da solução

Manter o menor preço já visto e atualizar o maior lucro possível.

Justificativa da estrutura

Array representa dias sequenciais permitindo varredura linear.

Código

```
def max_profit(prices):
    menor = float('inf')
    lucro = 0
    for p in prices:
        menor = min(menor, p)
        lucro = max(lucro, p - menor)
    return lucro
```

Complexidade

Tempo: O(n)
Memória: O(1)

Testes adicionais (2)

```
print(max_profit([7,1,5,3,6,4]))
print(max_profit([7,6,4,3,1]))
```

3. Contains Duplicate

Link: https://leetcode.com/problems/contains-duplicate/

Estrutura principal utilizada: Array + Set

Ideia da solução

Converter para Set e comparar tamanho.

Justificativa da estrutura

Set elimina duplicatas automaticamente com inserção O(1).

Código

```
def contains_duplicate(nums):
    return len(nums) != len(set(nums))
```

Complexidade

Tempo: O(n)
Memória: O(n)

Testes adicionais (2)

```
print(contains_duplicate([1,2,3,1]))
print(contains_duplicate([1,2,3,4]))
```

4. Maximum Subarray (Kadane)

Link: https://leetcode.com/problems/maximum-subarray/

Estrutura principal utilizada: Array

Ideia da solução

Decidir se inicia novo subarray ou continua acumulando.

Justificativa da estrutura

Problema depende de soma contínua sequencial.

Código

```
def max_subarray(nums):
    atual = maximo = nums[0]
    for i in range(1, len(nums)):
        atual = max(nums[i], atual + nums[i])
        maximo = max(maximo, atual)
    return maximo
```

Complexidade

Tempo: O(n)
Memória: O(1)

Testes adicionais (2)

```
print(max_subarray([-2,1,-3,4,-1,2,1,-5,4]))
print(max_subarray([-1,-2,-3]))
```

5. Product of Array Except Self

Link: https://leetcode.com/problems/product-of-array-except-self/

Estrutura principal utilizada: Array

Ideia da solução

Calcular prefixos e sufixos para cada posição sem usar divisão.

Justificativa da estrutura

Array permite cálculo posicional eficiente.

Código

```
def product_except_self(nums):
    n = len(nums)
    res = [1]*n
    prefixo = 1
    for i in range(n):
        res[i] = prefixo
        prefixo *= nums[i]
    sufixo = 1
    for i in range(n-1, -1, -1):
        res[i] *= sufixo
        sufixo *= nums[i]
    return res
```

Complexidade

Tempo: O(n)
Memória: O(1) extra

Testes adicionais (2)

```
print(product_except_self([1,2,3,4]))
print(product_except_self([-1,1,0,-3,3]))
```

6. Reverse Linked List

Link: https://leetcode.com/problems/reverse-linked-list/

Estrutura principal utilizada: Lista Encadeada

Ideia da solução

Inverter ponteiros iterativamente.

Justificativa da estrutura

Lista permite manipulação direta de ponteiros.

Código

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def print_list(head):
    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")

def reverse_list(head):
    anterior = None
    atual = head
    while atual:
        prox = atual.next
        atual.next = anterior
        anterior = atual
        atual = prox
    return anterior
```

Complexidade

Tempo: O(n)
Memória: O(1)

Testes adicionais (2)

```
# Teste 1
a = ListNode(1, ListNode(2, ListNode(3)))
print_list(reverse_list(a))

# Teste 2
b = ListNode(5)
print_list(reverse_list(b))
```

7. Merge Two Sorted Lists

Link: https://leetcode.com/problems/merge-two-sorted-lists/

Estrutura principal utilizada: Lista Encadeada

Ideia da solução

Comparar nós e anexar o menor.

Justificativa da estrutura

Lista permite reorganização sem copiar dados.

Código

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def print_list(head):
    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")

def merge_two_lists(l1, l2):
    dummy = ListNode()
    atual = dummy
    while l1 and l2:
        if l1.val < l2.val:
            atual.next = l1
            l1 = l1.next
        else:
            atual.next = l2
            l2 = l2.next
        atual = atual.next
    atual.next = l1 if l1 else l2
    return dummy.next
```

Complexidade

Tempo: O(n+m)
Memória: O(1)

Testes adicionais (2 obrigatórios)

```
# Teste 1
l1 = ListNode(1, ListNode(3))
l2 = ListNode(2, ListNode(4))
print_list(merge_two_lists(l1, l2))

# Teste 2
l1 = None
```

```
l2 = ListNode(5)
print_list(merge_two_lists(l1, l2))
```

8. Linked List Cycle

Link: https://leetcode.com/problems/linked-list-cycle/

Estrutura principal utilizada: Lista Encadeada

Ideia da solução

Usar algoritmo de Floyd (lento e rápido).

Justificativa da estrutura

Listas permitem ciclos via ponteiros.

Código

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def print_list(head):
    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")

def has_cycle(head):
    lento = rapido = head
    while rapido and rapido.next:
        lento = lento.next
        rapido = rapido.next.next
        if lento == rapido:
            return True
    return False
```

Complexidade

Tempo: O(n)
Memória: O(1)

Testes adicionais (2)

```
# Teste 1 (sem ciclo)
a = ListNode(1, ListNode(2))
print(has_cycle(a))

# Teste 2 (com ciclo)
b = ListNode(1)
b.next = b
print(has_cycle(b))
```

9. Remove Nth Node From End

Link: https://leetcode.com/problems/remove-nth-node-from-end-of-list/

Estrutura principal utilizada: Lista Encadeada

Ideia da solução

Criar distância entre ponteiros para remover nó correto.

Justificativa da estrutura

Permite ajuste eficiente de ponteiros.

Código

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def print_list(head):
    while head:
        print(head.val, end=" -> ")
        head = head.next
    print("None")

def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    lento = rapido = dummy
    for _ in range(n+1):
        rapido = rapido.next
    while rapido:
        lento = lento.next
        rapido = rapido.next
    lento.next = lento.next.next
    return dummy.next
```

Complexidade

Tempo: O(n)
Memória: O(1)

Testes adicionais (2)

```
# Teste 1
a = ListNode(1, ListNode(2, ListNode(3)))
print_list(remove_nth_from_end(a, 2))

# Teste 2
b = ListNode(1)
print_list(remove_nth_from_end(b, 1))
```

10. Valid Parentheses

Link: https://leetcode.com/problems/valid-parentheses/

Estrutura principal utilizada: Pilha

Ideia da solução

Empilhar aberturas e validar fechamentos.

Justificativa da estrutura

Pilha segue comportamento LIFO.

Código

```
def is_valid(s):
    pilha = []
    mapa = {')':'(', '}':'{', ']':'['}
    for c in s:
        if c in mapa:
            topo = pilha.pop() if pilha else '#'
            if mapa[c] != topo:
                return False
        else:
            pilha.append(c)
    return not pilha
```

Complexidade

Tempo: O(n)
Memória: O(n)

Testes adicionais (2)

```
print(is_valid("()[]{}"))
print(is_valid("(]"))
```

11. Min Stack

Link: https://leetcode.com/problems/min-stack/

Estrutura principal utilizada: Pilha

Ideia da solução

Usar pilha auxiliar para rastrear mínimo.

Justificativa da estrutura

Necessário LIFO e acesso ao mínimo em O(1).

Código

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []
    def push(self, val):
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)
    def pop(self):
        if self.stack.pop() == self.min_stack[-1]:
            self.min_stack.pop()
    def top(self):
        return self.stack[-1]
    def getMin(self):
        return self.min_stack[-1]
```

Complexidade

Tempo: O(1)
Memória: O(n)

Testes adicionais (2)

```
s = MinStack()
s.push(3); s.push(1); s.push(2)
print(s.getMin())
s.pop()
print(s.getMin())
```

12. Daily Temperatures

Link: https://leetcode.com/problems/daily-temperatures/

Estrutura principal utilizada: Pilha Monotônica

Ideia da solução

Usar pilha para resolver próximo maior elemento.

Justificativa da estrutura

Pilha armazena índices pendentes.

Código

```
def daily_temperatures(temperatures):
    res = [0]*len(temperatures)
    pilha = []
    for i, t in enumerate(temperatures):
        while pilha and temperatures[pilha[-1]] < t:
            idx = pilha.pop()
            res[idx] = i - idx
        pilha.append(i)
    return res
```

Complexidade

Tempo: O(n)
Memória: O(n)

Testes adicionais (2)

```
print(daily_temperatures([73,74,75,71,69,72,76,73]))
print(daily_temperatures([30,40,50,60]))
```

Como usei IA nesta atividade

IA utilizada: ChatGPT

Prompts utilizados:
1. "Explique solução O(n) para Two Sum."
2. "Explique algoritmo de Floyd."
3. "Gere 2 testes adicionais por problema."
4. "Revise complexidade de tempo e memória."
5. "Identifique edge cases importantes."

Reflexão:
A IA auxiliou na estruturação das soluções e validação de complexidade.
Todos os códigos foram revisados, adaptados e testados manualmente pela equipe.